



Trabajo Práctico N°1

ChompChamps

72.11 - Sistemas Operativos
2do Cuatrimestre de 2025

Grupo 1

Integrantes:

Ignacio Gastón Frund - 61465
Joaquín José Huerta - 64252
Thomas Adrianus Ruijgt - 62875

Índice

Decisiones tomadas durante el desarrollo.....	2
Instrucciones de compilación y ejecución.....	2
Inicialización del entorno.....	2
Compilación y ejecución.....	3
Limitaciones.....	4
Problemas encontrados durante el desarrollo y solución.....	4
Fuentes externas.....	5

Decisiones tomadas durante el desarrollo

Se comenzó consultando `man shm_open.3` en un sistema operativo Linux (específicamente WSL - Ubuntu) para implementar una primera versión de la memoria compartida indicada por la cátedra. A partir de esta se programaron un archivo `view.c` y otro `player.c` que usen el binario otorgado por la cátedra para probar su funcionamiento. Convencidos de que estos cumplieran el mínimo necesario, comenzó el desarrollo de `master.c` ya que es la parte más compleja del proyecto; inicialmente con una cantidad fija de 5 turnos. Desde este punto el desarrollo se distribuyó entre los tres procesos, priorizando el proceso `master`.

En `master` se decidió dividir algunas de sus tareas en otros archivos para facilitar la comprensión del código: primero para la memoria compartida, luego para controlar a sus hijos. De esta manera la creación, envío y recepción de información, uso de semáforos y final “terminación” de cada proceso se hace en sus respectivos archivos. Se conservó en `master` aquello que involucra a todos los procesos o exclusivamente al proceso `master`, como puede ser el scheduler round robin entre los `players` o el administrador de señales.

El scheduler utiliza un set de file descriptors que en cada turno contiene aquel del próximo player a moverse. Se le informa al mismo que puede moverse, pero si no envió una solicitud en el timeout determinado, no se le mueve y se continúa con el ciclo. Luego se imprime el estado, se espera el delay y se selecciona al siguiente player.

Finalizado el scheduler, se le pide a los procesos que salgan y se destruyen los semáforos y la memoria compartida.

En el caso de los hijos, ambos validan los parámetros por si acaso y abren las memorias compartidas. Al finalizar sus ciclos, desmapean sus referencias a las memorias compartidas para que `master` pueda eliminarlas.

En su ciclo, `view` espera a que `master` indique que hay algo para imprimir en el respectivo semáforo y al finalizar le indica que ya finalizó la impresión con otro semáforo.

En los ciclos de los `players`, se aborda el problema de sincronización de lectores y escritores como se vió durante la materia. Cada uno guarda el estado para no desperdiciar tiempo de uso de recursos compartidos y luego analiza su movimiento. Si no llega a algún movimiento válido, se autobloquea. Al finalizar su ciclo, además de desmapear la memoria compartida, cierra su file descriptor de escritura.

Finalmente, para la implementación de la inteligencia artificial del player, se decidió que si bien se tiene que tener en cuenta cuantas recompensas obtiene este a la hora de ejecutar un movimiento, también se consideró importante que este sea consciente de la cantidad de libertad, ya que este en su afán por obtener muchos puntos rápidamente, es decir,

una recompensa “inmediata”, podría encerrarse y perjudicarse, por lo que también, a la hora de decidir hacia donde ir, también considera su “libertad” por medio de una implementación similar a BFS.

Instrucciones de compilación y ejecución

Para ejecutar correctamente el proyecto, es necesario tener la imagen de docker provista por la cátedra, [agodio/itba-so-multi-platform:3.0](#). A su vez, es necesario tener un clon del repositorio. Adicionalmente, es opcional tener instalado PVS-Studio y contar con una licencia para poder ejecutar un comando particular del makefile.

Inicialización del entorno

Para inicializar el entorno, utilizar un contenedor con la imagen especificada o correr el siguiente comando:

```
docker run --rm -v ${PWD}:/root --security-opt seccomp:unconfined -it agodio/itba-so-multi-platform:3.0
```

Posteriormente, es necesario moverse al directorio root, lo cual se puede hacer mediante el siguiente comando inmediatamente después de ejecutado el primero:

```
cd /root
```

Compilación y ejecución

El proyecto cuenta con un makefile el cual sirve para facilitar la tarea de compilación. En el archivo [README.md](#) se encuentran todas las instrucciones disponibles, por lo que aquí se especificará el compilado y ejecutado como si fuera la primera vez.

Para el compilado se debe ejecutar el siguiente comando:

```
make all
```

Luego de ejecutar este comando, se generan 3 binarios, uno correspondiente al juego ([ChompChamps](#)), uno correspondiente al jugador ([player_g1](#)) y otro correspondiente a la vista ([view_g1](#)).

Para ejecutar el juego, se debe correr el archivo **ChompChamps** junto con los siguientes parámetros.

- **'-p'**: Ruta/s de los binarios de los jugadores. Mínimo: 1, Máximo: 9
- **[-w]**: Ancho del tablero. Default y mínimo: 10
- **[-h]**: Alto del tablero. Default y mínimo: 10
- **[-d]**: milisegundos que espera el máster cada vez que se imprime el estado. Default: 200
- **[-t]**: Timeout en segundos para recibir solicitudes de movimientos válidos. Default: 10
- **[-s]**: Semilla utilizada para la generación del tablero. Default: time(NULL)
- **[-v]**: Ruta del binario de la vista. Default: Sin vista.

Todos los parametros son opcionales excepto el parámetro **-p**.

Ejemplos de uso

```
./ChompChamps -p player_g1 -v view_g1
```

Este ejemplo crea una partida con un solo jugador utilizando la vista compilada.

```
./ChompChamps -p player_g1 player_g1 -v view_g1 -w 20 -h 20
```

Este ejemplo crea una partida con dos jugadores, utilizando un tablero de 20 x 20, con la vista compilada.

Limitaciones

El proyecto contó con limitaciones tanto durante el desarrollo como en su implementación. En cuanto a las primeras, si bien se utilizó como herramienta para debugging, se utilizó la versión gratuita de PVS-Studio, puede que la versión completa haya identificado más errores que no se pudieron identificar con la prueba gratis.

En cuanto a las limitaciones de la versión implementada, la vista puede ser afectada por el tamaño de la consola donde se corra. El ejemplo más notorio es cuando se encoge lo suficiente al punto que el **"GAME OVER"** impreso al final se deforma.

La inteligencia artificial del jugador, si bien se intentó que sea lo más abarcativa posible, da lugar a mejoras. La implementación actual por más que toma decisiones "a corto y mediano plazo", las toma sin un rumbo fijo, el cual podría implementarse por ejemplo, caminando

siempre hacia las celdas con mayor puntaje y luego aplicar los criterios actuales mientras camina hacia esta.

Problemas encontrados durante el desarrollo y solución

El principal problema fue dado a la hora de comenzar el desarrollo, debido a que el equipo no tenía conocimiento de cómo arrancar. Esto fue consultado durante clase donde se recomendó comenzar por la creación de una memoria compartida. Esta fue implementada gracias a la bibliografía del manual, en particular el ejemplo disponible de la página [shm_open\(3\)](#). Una vez implementado el ejemplo, se pudo continuar el desarrollo, reemplazando parte por parte hasta lograr la versión final de la implementación.

Otro problema fue encontrado a la hora de utilizar PVS-Studio para encontrar errores. No solo hubo una curva de aprendizaje sino que, parecía no funcionar, lanzando errores fatales, mostrando el informe vacío. Este sin embargo tuvo una resolución sencilla pero que no fue fácil de darse cuenta. Esto era causado por la falta de la inclusión de las líneas especificadas en la página oficial para su uso mediante una licencia gratis. Una vez incluidas, el funcionamiento fue impecable.

Otro problema que se encontró fue que, con el afán de proteger la memoria compartida, se estableció como flag *O_EXCL* para la creación de la memoria compartida por parte del máster. Esto causa que si ya existe una con ese nombre, se retorna un error. Sin embargo, si bien en papel funcionó bien, siempre que el programa se interrumpiera antes que su finalización “natural”, se tenía que manualmente borrar la memoria compartida. Si bien esto podría haberse solucionado con la implementación del *makefile*, donde se le pide que borre estas, no se consideró como una solución realmente efectiva, ya que simplemente esconde el problema. Por tanto, se decidió implementar un signal handler que se encargue de liberar los recursos, entre ellos la memoria compartida. Luego de que este funcione, se decidió cambiar el flag por uno más laxo, *O_TRUNC*, el cual si existiera la memoria, en lugar de retornar error simplemente la trunca a 0 bytes. Este handler a su vez facilitó el manejo de los otros recursos más allá de la memoria compartida en caso de recibir una señal, como por ejemplo, esperar a los procesos de los jugadores y la vista más fácilmente.

Rutas relativas de la vista y jugador

- Vista: `./view_g1`
- Jugador: `./player_g1`

Fuentes externas

Página del manual con ejemplo utilizado como base del proyecto:

https://man7.org/linux/man-pages/man3/shm_open.3.html

Páginas del manual de Linux utilizadas para la comunicación y administración de procesos:

`man execve.2`

`man open.2`

`man close.2`

`man read.2`

`man write.2`

`man sem_wait.3`

`man kill.2`

`man wait.2`

`man mprotect.2`

`man fork.2`

De muchas de estas se tomaron ejemplos textuales. Algunos de estos fueron mejorados con prácticas más seguras o modernas.